

Blockchains Coursework

B217645

November 2025

1 Introduction

A classical sealed-bid Vickrey auction operates as a two-stage process. In the first stage, bidders submit their bids privately and confidentially. In the second stage, all bids are revealed, and the highest bidder is declared the winner. Crucially, the winning bidder pays an amount equal to the second-highest bid, rather than their own. To protect sellers, a reserve price may be established, representing the minimum acceptable sale price.

From the perspective of game theory, the dominant strategy for each bidder is to bid their true valuation of the item [1]. This mechanism ensures fairness in multiple ways: the seller either achieves a satisfactory price or retains the item, no bidder risks overpaying, and the winning bidder benefits by paying only the second-highest bid. Consequently, Vickrey auctions are widely regarded as an efficient and equitable auction format, minimizing the strategic complexity for participants while promoting truthful bidding behavior.

Translating the Vickrey auction format into a Solidity smart contract for auctioning NFTs presents a unique set of challenges. It requires a careful balance between user experience and the security requirements inherent to smart contracts. Decisions must be made regarding the prioritization of security, efficiency, and fairness. Additionally, considerations such as gas optimization and ease of use are critical to ensure the contract is practical and accessible for users. This report evaluates a sealed-bid Vickrey auction that I developed and deployed, assessing its effectiveness, security, gas efficiency, and user experience. It also examines a similar auction created by a classmate for comparison, highlighting different approaches to implementation and optimisation.

2 Contract Overview

The Vickrey auction contract is formed of the following:

3 Structs: *Auction, AuctionInfo, AuctionResults*

10 events: *AuctionCreated, BidPlaced, BidRevealed, AuctionEnded, Withdrawn, NFTClaimed, NFTReclaimed, DepositForfeited, ForfeitedClaimed, UnclaimedDepositRecovered*

9 Modifiers: *onlySeller, onlyDuringBidding, onlyDuringReveal, onlyAfterReveal, onlyIfNFTNotClaimed, onlyAfterAuctionEnded, auctionExists, onlyDuringClaim, onlyAfterClaim*

9 Key Functions: *createAuction, placeSealedBid, reveal, endAuction, withdraw, claimNFT, reclaimNFT, claimForfeitedBatch, recoverUnclaimedDeposit*

1 Pure Function: *computeHash*

7 Getter Functions: *getAuctionInfo, getAuctionResults, getBidHash, getDeposit, hasRevealed, hasForfeited, getBidders*

Structs organise all auction data into clear format 'structures', allowing the contract to manage multiple auctions efficiently. The key functions then implement the full Vickrey auction flow whilst ensuring the process remains secure and fair. Modifiers enforce essential rules such as access control and phase restrictions in a clean, reusable way, improving readability and gas efficiency by reducing duplicated checks. Events provide transparency by broadcasting important state changes to external users and interfaces, allowing participants to follow the auction without constant on-chain queries. The pure hashing function lets bidders generate their bid commitments off-chain at zero gas cost, preserving bid privacy. Finally, getter functions

offer safe, read-only access to auction information, improving usability without affecting security. Together, these components create a secure, efficient, and transparent auction system.

An auction is begun by calling *createAuction*. The seller specifies the NFT they are auctioning along with the reserve price, bidding period length, revealing period length and the initial deposit they want bidders to pay. The auction is then given a unique identifier, the NFT is transferred to the contract and the auction is emitted to the blockchain.

At any point whilst the auction is live a user can call *getAuctionInfo* to find the vital information needed to participate in the bid.

Then begins the bidding period. Bidders may only submit one bid and they are able to get an encrypted hash of their bid by calling *computeHash* off-chain. They can then submit this hash as a commitment to the auction with the function *placeSealedBid*. As the encrypted hash was computed off-chain, this ensures that their bid remains secret to everyone, including the seller, until the reveal period. A deposit is also sent to the contract with this function.

After the bidding period, the revealing period begins in which bidders call *reveal* with their original bid and nonce; the contract verifies that the hash matches the prior commitment, ensuring authenticity and linking the bid to a specific bidder. At this stage the remainder of the bid amount is sent to the contract.

As bids are revealed, the *highestBid*, *highestBidder* and *secondHighestBid* variables are updated as needed. In the case of a tie, priority is given to the bidder that revealed their bid first.

Once this period has finished, any user can call *endAuction* which emits the *auctionId* alongside the *highestBid*, *highestBidder* and *secondHighestBid* (the amount they will pay for the NFT). This finalises the auction and means the function *getAuctionResults* can now be called to find out the results of the auction in addition to the info at setup.

At this point sellers and bidders are able to call *withdraw*, a "payable" function which returns the correct amount of funds to the seller, winner and losing bidders depending on the outcomes of the auction. The winner can also claim their NFT by calling *claimNFT*. However, if there were no valid bids and thus no winner, the seller can reclaim their NFT by calling *reclaimNFT*.

Upon conclusion of the revealing period, the claim period begins. This is a period in which the seller is able to claim any forfeited deposits for themselves by calling *claimForfeitedBatch*. However, if they fail to reclaim these deposits in this period, they lose that opportunity and the sellers can reclaim their deposits by calling *recoverUnclaimedDeposit*.

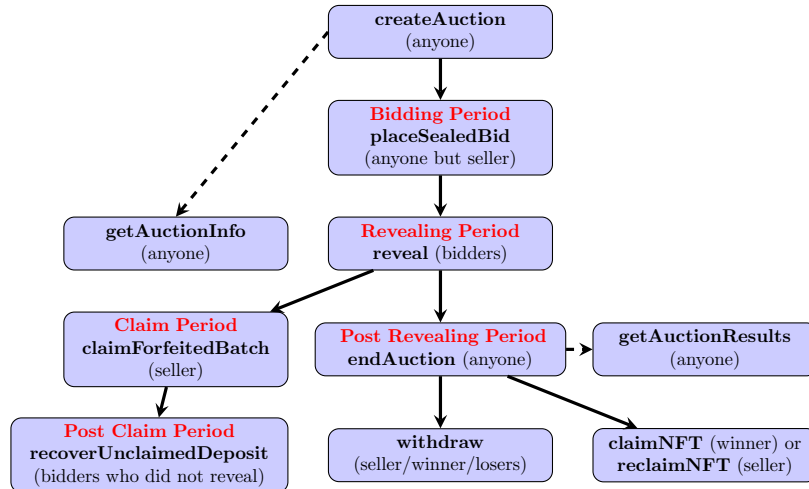


Figure 1: Lifecycle of a Vickrey auction in the contract, showing each period, the relevant functions and who can call those function.

3 Security Analysis

Security is a core issue when deploying contracts onto the Ethereum blockchain. If a contract isn't secure, then malicious participants can exploit the contract and skew the favours in their odds. The result is an

auction that is inherently unfair and therefore unusable. Hence, in this contract, security is treated with upmost priority. In this section we explore some of the key security concerns faced by the contract and how these concerns have been mitigated. We also explore any instances where we have sacrificed security protocols in favour of performance and user experience.

3.1 Denial of Service: Unbounded Operations

This happens when an attacker prevents honest users from interacting with a contract by exploiting operations that have no fixed limit. The result is an unusable contract. This attack commonly arises from functions that iterate over dynamic arrays or mappings without bounds, causing the gas usage to grow with the number of elements. In this contract, the *claimForfeitedBatch* function iterates over a list of bidders for a specific auction, allowing the seller to claim any forfeited deposits. The risk of exceeding the block gas limit is mitigated by the "maxBatch" parameter, which limits the number of bidders processed per transaction.

Even if a seller were to act maliciously by setting a very high "maxBatch" value, the contract remains secure. If the gas limit is exceeded, only that transaction fails as no other functions in the contract are dependent on it being successfully called. Hence, all other functions in the same auction, such as *withdraw* or *claimNFT*, remain callable. Moreover, since each auction has its own auction ID, this issue does not affect other auctions or other sellers ability to call *claimForfeitedBatch*.

Finally, because only the seller is able to call this function, any failed batch attempt only impacts the seller of that specific auction, potentially delaying or preventing them from claiming forfeited deposits within the allowed claim period. This strongly deters sellers from setting the "maxBatch" parameter too high and ensures the auction remains secure and functional for all participants, while providing a controlled mechanism to process potentially large bidder lists efficiently.

3.2 Denial of Service: Griefing

A griefing attack occurs when an adversary deliberately incurs some cost, such as gas fees or deposits, in order to make the contract expensive or inconvenient for others to use. The intention is to cause disruption rather than to make a profit. In this contract, the seller is the most likely target of such an attack. An attacker could, for example, create many bids using a large number of Sybil addresses to inflate the list of bidders, increasing the gas cost for the seller when claiming forfeited deposits. The contract defends against this by restricting each address to a single bid, meaning an attacker would need to control or collude with many addresses to have any real impact. Since each bid also requires an initial deposit, such behaviour would quickly become expensive. Moreover, every time a bid is placed or revealed, an event is emitted — *BidPlaced(auctionId, bidder)* when a bid is placed and *BidRevealed(auctionId, bidder, amount)* when a bid is revealed. Using these events, the seller can efficiently identify non-revealing bidders off-chain by creating two sets: one containing all bidders who placed bids, and another containing all bidders who revealed. The difference between these two sets gives the exact list of bidders who failed to reveal. This allows the seller to target only those addresses when calling *claimForfeitedBatch*, avoiding the need to iterate through the entire list of bidders on-chain.

Economic incentives also discourage griefing behaviour. Any bidder who refuses to reveal forfeits their deposit, meaning that an attacker attempting to flood the auction only ends up transferring value to the seller. The seller, in turn, is protected from exceeding the block gas limit when claiming forfeits through the inclusion of a "maxBatch" parameter, which allows forfeited deposits to be processed in manageable batches. As a result, any attempt to grief the auction becomes both economically and technically ineffective.

A griefing attack is where an attacker deliberately incurs some costs (gas, deposits, etc.) in order to make the contract expensive or inconvenient for others. The goal is to cause disruption rather than to make a profit. In this contract, the seller is most likely to be targeted by this attack. An attacker could place many bids (for example, from many Sybil addresses) in order to bloat the list of bidders, increasing the gas cost for the seller when claiming forfeited deposits. The contract defends against this by restricting each address to a single bid, meaning an attacker would need to control or collude with many addresses to have any real impact. Since each bid also requires an initial deposit, such behaviour would quickly become expensive. Moreover, every time a bid is placed or revealed, an event is emitted: *BidPlaced(auctionId, bidder)* when a bid is placed and *BidRevealed(auctionId, bidder, amount)* when a bid is revealed. The seller can then,

off-chain, create a set of bidders and bidders who have revealed their bids, and then compute the difference between these two sets to gain a list of all bidders who have not revealed their bids. Then the seller can target only those addresses when calling *claimForfeitedBatch*, avoiding the need to iterate through the entire list of bidders on-chain.

Economic friction prevents the attacker(s) from not revealing their bids as in doing so they forfeit their deposits, which in turn means more Ethereum going to the seller. As explained previously, the "maxBatch" parameter protects the seller from exceeding the gas block limit, no matter the number of forfeited deposits. As a result, any attempt to grief the seller in this way becomes expensive and ineffective.

Griefing could also be caused by a seller refusing to transfer their NFT to the auction winner, or a winner refusing to pay for or claim the NFT, thereby preventing the auction from being finalised. However, the contract's design prevents such scenarios. The auction can be concluded as soon as the reveal period has ended by the *endAuction* function which anyone can call. Honest participants are incentivised to call this function because they cannot claim their winnings or deposits until the auction is ended, while attackers have no means to prevent its execution. Furthermore, each auction operates under a unique *auctionId*, meaning that auctions are entirely independent of each other. Even if one auction experienced disruptive behaviour, it would not affect any others. Finally, all assets are held in escrow by the contract itself. The seller must transfer their NFT to the contract before the auction begins, and bidders must send the value of their bids when revealing. Because both assets are already under the control of the contract, neither party can refuse to hand over what they owe. Together, these mechanisms ensure that neither the seller nor the winning bidder can delay settlement or obstruct the auction's conclusion, preserving fairness, efficiency, and reliability throughout the auction process.

3.3 Reentrancy

Reentrancy attacks occur when a malicious contract repeatedly calls back into a vulnerable contract during an external call. This typically exploits the fact that the vulnerable contract's state has not yet been fully updated, allowing the attacker to drain funds or manipulate the contract's state. It is a potential issue whenever funds or tokens are being transferred to or from the contract. This contract protects itself against such attacks by applying a reentrancy guard ("nonReentrant") to every function that withdraws funds or tokens from the contract. These are *withdraw*, *claimNFT*, *reclaimNFT*, *claimForfeitedBatch* and *recoverUnclaimedDeposit*.

In addition, the contract follows the Checks-Effects-Interactions (CEI) pattern. This means that each potentially vulnerable function first validates the input data and the caller, then updates the internal contract state to reflect the required changes, and only then interacts with external contracts or users. For example, in the *claimNFT* function, the contract first checks that the selected auction exists, that it has ended and that the NFT has not already been claimed. It then verifies that the caller of the function is indeed the highest bidder. After performing these checks, it then sets the *nftClaimed* flag "True" and only then transfers the NFT to the winner of the auction. By updating the contract state before making any external calls, it ensures that even if a reentrant call were attempted, it would have no effect.

3.4 Front-Running

A front-running attack involves an attacker observing a pending transaction on the blockchain, such as a bid submission, and submitting their own transaction with higher gas fees so that it is mined first. In an auction setting, this would allow them to adjust their bid after seeing another bidder's submission, potentially manipulating the outcome.

The contract utilises a commit-reveal design to defend against this. Bids are committed as hashed commitments, which tie a bidder to their bid whilst hiding the value of the bid until the bidding phase is complete. During the bidding phase nobody is aware of anyone else's bids, so there is no risk of a front-running attack. Bids are only revealed after this period, by which time bids cannot be adjusted. While the contract remains theoretically vulnerable to front-running during the reveal phase, this is a deliberate design choice, discussed further in section 3.8.

3.5 Overflow and Underflow

Arithmetic overflow and underflow are no longer a direct issue in Solidity 0.8.0 and above, as the compiler automatically reverts transactions when they occur instead of wrapping around to zero or the maximum number. However, it is still important to design functions to avoid unnecessary overflows or underflows to ensure smooth and efficient operation.

In this contract, counters such as *auctionCount* are stored as "uint256", which effectively prevents them from ever overflowing in practice. Furthermore, automatic underflow checks are utilised in the withdraw function to enforce correctness. When the highest bidder's deposit is set to zero after their first withdrawal, any subsequent attempt to withdraw causes the calculation *deposit* - *secondHighestBid* or *deposit* - *reservePrice* to underflow. This triggers an automatic revert, effectively blocking a second withdrawal. This mechanism ensures that each participant can only claim their funds once, while relying on built-in underflow protections rather than requiring additional manual checks.

3.6 Stealing Deposits

A malicious seller could attempt to exploit the contract by forcing honest bidders to forfeit their deposits. For example, they might set an excessively long bidding period to attract many bids, followed by an extremely short reveal period, causing all bids to be forfeited. This would unfairly allow the seller to reclaim their NFT while simultaneously collecting the deposits from all bidders.

The contract mitigates this risk in the *createAuction* function by enforcing minimum durations for both bidding and reveal periods, and by requiring that the reveal period is at least 25% of the bidding period. Hence, a four day bidding period would be followed by a 1 day revealing period. This ensures bidders have sufficient time to reveal their bids while preventing excessive locking of funds, as bidders must deposit the remaining value of their bid in the reveal stage. In a future version of the contract, bidders would be able to withdraw their funds as soon as they have revealed their bid, provided they are not the highest bidder. The claim period for forfeited deposits is set equal to the reveal period, giving the seller a fair window to claim forfeited funds without risking loss due to timing constraints.

Finally, the *getAuctionInfo* function can be called off-chain at no gas cost and allows users to view key auction parameters. This includes the bidding and revealing periods as well as the required deposit. It reduces the risk of manipulation as bidders are able to make informed decisions on whether to participate in an auction. These measures collectively ensure that bidders' deposits are protected while maintaining fairness and efficiency in the auction process.

3.7 Other Attacks

The contract is designed to be secure against many other issues that could cause the function to fail or make the auction unfair. Strategic withholding, where a bidder refuses to reveal their bid to save gas because they cannot win, is discouraged by the deposit mechanism: any unrevealed bids forfeit their deposit. Fund locking, such as a seller refusing to finalise an auction, is prevented because the auction can be ended by anyone once the reveal period ends. Honest parties are incentivised to end the auction so that they can withdraw their funds. Incorrect claims are prevented by strict access control and checks. For example, only the highest bidder can claim the NFT, or only the seller can claim it if no highest bid exists. Sellers are also prevented from locking away deposits indefinitely, as bidders can reclaim unclaimed deposits once the claim period expires.

3.8 Vulnerabilities and Tradeoffs

Security is at the forefront of this contract, as reflected in the design tradeoffs made. That said, the reveal period remains somewhat vulnerable to front-running. In the event of a tie during the reveal period, the first bidder to reveal takes priority. This means that if a user observes another bidder submitting the same amount, they could attempt to reveal their bid with a higher gas fee to gain priority and become the highest bidder. However, this risk is relatively small, as it is uncommon for two bidders to place exactly the same amount, and even less common that these bids are the highest overall. Additionally, prioritizing the first

bidder incentivises participants to reveal their bids promptly, reducing the likelihood that users forget and accidentally forfeit their deposits, which supports the overall fairness and efficiency of the auction.

The commit-reveal scheme is central for the security of the contract, as it ensures bid-privacy and mitigates against front-running. However, it requires bidders to call two separate functions increasing gas costs and slightly complicating participation. This two-step process also spreads the submission of funds over two occasions, adding overhead, but it provides the benefit that bidders do not have to lock up their entire bid during the bidding period. Furthermore, should an honest bidder forget to reveal, they lose only their deposit and not their whole bid. [maybe this should be another section]. The submission of an initial deposit, as opposed to sending all funds in the reveal period, provides important protection against griefing and strategic withholding at the cost of making users lock up funds. This demonstrates a balancing of security with user experience and market fluidity.

The “maxBatch” parameter mitigates denial-of-service attacks caused by unbounded loops by limiting the number of forfeited deposits processed in a single transaction. While this adds some complexity and slightly increases gas costs, it allows the seller to efficiently claim multiple deposits without hitting the block gas limit. Having the seller claim forfeited deposits individually for each bidder who failed to reveal would be more secure, but it would drastically increase the number of transactions, gas costs, and the overall time required to complete the process. This approach could also complicate the implementation of the claim period, which is crucial to ensure that forfeited deposits are not locked away indefinitely .

4 Gas Evaluation

4.1 Gas Usage Testing

To analyse gas usage, I conducted two auctions involving one seller, denoted **A**, and five bidders, denoted **B**, **C**, **D**, **E**, and **F**. In the first auction, all bidders placed bids, but only **B**, **C**, and **D** revealed their bids. Seller **A** then ended the auction and claimed **E**’s forfeited deposit. Bidder **F** recovered their forfeited deposit, and bidders **B**, **C**, and **D** withdrew their funds. Finally, **D** claimed the NFT as the winner.

In the second auction, all bidders placed bids, but none revealed them. Bidder **B** ended the auction, after which seller **A** claimed the forfeited deposits of all bidders and reclaimed the NFT.

The exact gas execution costs for each function for both Auction 1 and Auction 2 can be found in the Appendix A. Table 1 shows the total gas used the seller and each bidder in each of the auctions.

Table 1: Total Gas Usage per Participant in Each Auction

Participant	Auction 1 Gas	Auction 2 Gas
A (Seller)	312,254	475,062
B	125,373	134,209
C	118,648	80,779
D	185,650	80,779
E	80,779	80,779
F	132,325	80,779

4.2 Contract User Fairness

From Table 1 and Appendix A, two observations are immediately clear. First, sellers generally face higher gas costs than bidders. Second, the cost of calling functions can vary depending on who calls them and when. This raises the question of user fairness: participants should feel that interacting with the Vickrey auction is equitable, including with respect to gas costs

It is reasonable for sellers to incur higher gas costs. They are the ones initiating the sale, and these costs can be considered a fee to list the NFT on the Ethereum blockchain. The *createAuction* function is the most expensive in the contract, aside from potentially *claimForfeitedBatch*. This is because it initializes multiple storage variables, such as *reservePrice*, *initialDeposit*, *endOfBidding*, *endOfRevealing*, and *forfeitClaimDeadline*. It also transfers the NFT to the contract, which a necessary cost for security purposes. While the first auction is slightly more expensive than subsequent auctions (because updating *auctionCount* from zero to

non-zero is more costly than updating a non-zero value), this difference is minor. Starting *auctionCount* at 1 could mitigate this small gap.

Another significant gas cost for the seller occurs when claiming forfeited deposits. The more deposits to claim, the higher the cost. The "maxBatch" parameter allows the seller to control these costs, maintaining fairness by enabling them to recover lost deposits while keeping gas use manageable. Overall, the higher gas costs for sellers are justified: they enable NFT sales and access to forfeited deposits.

For bidders, gas costs also vary. The first bidder pays more in *placeSealedBid* because the bidders array is initialized, whereas later bidders simply append to an existing array. The array is of importance in regards to transparency of the auction and so should not be removed. Moreover, this difference is typical in Solidity and is unlikely to cause concern.

Gas costs in reveal depend on the bid amount. Bids below the *reservePrice* or the current *secondHighestBid* incur minimal gas, as they do not update any auction state. Higher bids that affect *highestBid*, *highestBidder*, or *secondHighestBid* cost more, with the first revealer of a new highest bid paying the most due to multiple storage writes. Although initializing these variables to non-zero values could reduce gas, it would complicate the auction logic and impose constraints on deposit and reserve price settings. Instead, the design choice to incentivise early revealing through priority in case of ties paired with the risk of forfeiting deposits for late reveals aims to balance fairness and efficiency.

Gas costs for withdraw differ by participant type. Losing bidders simply retrieve their deposits via the deposit mapping, while winners pay slightly more, as their refund accounts for the *secondHighestBid* or *reservePrice*. Sellers incur the highest gas because their payout calculation considers the *highestBid*, *secondHighestBid*, and ensures that *sellerWithdrawn* is updated. In this way, sellers pay the most, winners slightly less, and losing bidders the least, aligning logically with each participant's role in the auction.

In conclusion, the contract's gas structure is fair. Sellers bear higher costs in exchange for facilitating the auction and recovering forfeited deposits, winners pay more due to their influence on auction outcomes, and losing bidders incur minimal costs. Minor gas discrepancies for early bidders or revealers are justified and can be incentivized through contract mechanics. Overall, the design ensures both fairness and efficiency for all participants.

4.3 Techniques for Fairness and Efficiency

Whilst achieving perfect fairness in a smart contract is inherently subjective and often in tension with security or user-experience considerations, the contract employs a number of techniques to ensure it runs as fairly and efficiently as possible.

Most notably, nearly every function begins with require statements that perform early reverts when invariant conditions are violated. This is important not only for security but also for gas fairness, because an early revert refunds unused gas instead of executing unnecessary logic. For example, the *reveal* function immediately checks that the caller has placed a bid and has not already revealed it, preventing gas wastage.

The *placeSealedBid* function restricts bidders to a single bid per auction. In addition to mitigating griefing attacks such as spam-bidding, this significantly simplifies the rest of the contract and saves gas. Supporting multiple bids would require additional storage, complex mappings, and additional validation logic. This would increase gas consumption for everyone.

In functions such as *withdraw*, certain variables, which are only needed in that specific function, are defined locally rather than in storage. This saves gas because local variables live in stack or memory, which are orders of magnitude cheaper than persistent storage. Avoiding SLOAD and SSTORE operations prevents expensive storage access. The *withdraw* function also zeroes out each caller's deposit when they complete withdrawal, producing a small gas refund due to the storage slot being cleared.

If a function exceeds the block gas limit, the entire call reverts and all gas is lost. The "maxBatch" parameter in *claimForfeitedBatch* mitigates against this as processing forfeited deposits in user-specified batches avoids this failure. Although batching requires multiple calls, it prevents a single oversized call from wasting potentially enormous amounts of gas. This is an especially important efficiency consideration in auctions with many bidders. Further, within *claimForfeitedBatch* the contract checks that each bidder in the provided list both failed to reveal and still has a non-zero deposit that has not already been forfeited. These validation steps prevent the function from attempting to process already-claimed or zero-valued deposits, thereby avoiding unnecessary storage operations and wasted gas.

Functions in the contract are deliberately designed to perform only a single conceptual task. “Large” functions that attempt to accomplish several operations in one call often create redundant work and additional storage writes, increasing gas costs for users. By ensuring that each function does precisely what the caller intends gas consumption remains predictable and fair.

Fairness is ensured by getter functions, which provide off-chain access to auction state at zero gas cost. This allows users to query the contract without risking accidental gas expenditure. For example,

Getter functions provide off-chain access to auction state without any gas cost. This is an important fairness property because it allows bidders and sellers to query the contract (e.g., checking whether they have already revealed) without risking accidental gas expenditure. For example, if a bidder forgets whether they have revealed their bid, the *hasRevealed* function allows them to check without potentially wasting gas by calling the *reveal* function.

Events also avoid redundant parameters. For instance, the *BidPlaced* event does not include the deposit amount because it is fixed for all bidders. Emitting fewer parameters directly reduces event-logging gas costs, which helps keep bidding inexpensive.

Mappings related to each auction are stored inside the Auction struct rather than as separate top-level mappings. When mappings are defined inside a struct, they all derive their storage keys from the single storage slot that corresponds to that struct’s root. In contrast, if each mapping were declared at the top level, Solidity would assign each one its own dedicated storage slot, increasing the number of storage roots stored directly in the contract’s state. By nesting the mappings inside the struct, the contract needs only one top-level slot for the entire set of bidder-related mappings instead of four. This reduces the total number of storage slots reserved at deployment time, which in turn decreases the size of the deployed bytecode and lowers the contract’s deployment gas cost.

Finally, the ordering of fields within the Auction struct is chosen to maximise storage packing. A storage slot is 32 bytes, and Solidity packs smaller variables together when possible. By placing the address *highestBidder* next to the three bool values (*auctionEnded*, *nftClaimed* and *sellerWithdrawn*), these variables can share slots rather than each using separate ones. Correct packing can save up to three storage slots compared to a poorly ordered struct, thus lowering deployment and runtime gas usage whenever these fields are read or written.

4.4 Tradeoffs

Although auctions in the contract are independent, the existence of multiple concurrent auctions can cause gas costs to increase for certain operations. This is because, as more auctions are created, the contract occupies additional storage on the blockchain. Ethereum’s gas model charges slightly more when updating or interacting with storage in larger contracts, due to the underlying cost of SSTORE and SLOAD operations. However, this inefficiency is a worthwhile tradeoff. The contract’s ability to host multiple simultaneous auctions ensures that users do not have to wait for previous, potentially long, auctions to finish before they create a new one. Limiting the contract to a single active auction would not only reduce efficiency but could also introduce unfair dynamics, as users would race to call the next auction immediately after the previous one concludes.

All functions emit events upon completion. Emitting an event in this contract typically costs a few thousand gas, which is a relatively small amount compared to total function costs. Events are important as they increase transparency and serve as the primary mechanism for front-end interfaces to display the current state of the contract. This is because they can listen for these events without repeatedly querying the blockchain, reducing off-chain resource usage and unnecessary gas consumption.

The contract utilises uint256 variables throughout. Whilst gas could be saved by using smaller types, such as uint32 for *auctionCount*, these savings are minimal unless the variables can be packed together in the same storage slot. Opportunities for such packing are limited in this contract, and changing variable sizes could complicate the logic of interdependent variables, potentially making the contract harder to understand and maintain. Thus, the choice to use uint256 prioritizes readability/usability over marginal gas savings.

The *withdraw* function is one of the few functions within the contract that serves multiple different users depending on their role in the auction. Although most other functions optimise for gas efficiency by targeting a single user and action, it is more user-friendly to have a unified withdraw function for deposit funds. This design simplifies the user experience, as participants do not need to determine which variant of the function

to call, particularly if they are unsure whether they won the auction. The slightly increased gas cost of a single multi-purpose function is justified by the improvement in usability and clarity.

5 Peer’s Contract Evaluation

In this section, relevant code snippets can be found in Appendix A and are referred to by their "Listing" number.

5.1 Contract Overview

In this contract, a seller starts an auction by calling *createAuctionListing*, initiating the bidding period during which users submit hashed bids via *bid* without needing to lock a deposit. After the bidding period ends, only the seller can call *completeAuction*, which begins the reveal period. During this period, bidders reveal their bids by calling *revealBid* and sending their bid amounts. Once the reveal period concludes, either the seller or the winning bidder can finalise the auction using *finaliseAuction*, which transfers the NFT and distributes the payment and any refunds between the two of them. Only after this can other bidders retrieve their funds by calling *getRefund*. The contract also relies on internal functions such as *resetBids*, *hasBidder*, and *transferFund*, as well as getter functions, to support the key auction logic.

5.2 Secureness, Efficiency and Fairness

The peer contract demonstrates a reasonable degree of security, fairness, and efficiency. It employs *non-Reentrant* guards in all functions that interact with funds and could be vulnerable to reentrancy attacks, including *bid*, *completeAuction*, *revealBid*, *finaliseAuction* and *getRefund*. The combination of bid-commit setup of the auction and hashed bid submissions ensures the secrecy of bids until the reveal stage, providing protection against premature information disclosure. Whilst the use of *abi.encodePacked(bidAmount, secret)* could theoretically result in hash collisions, such collisions are unlikely in practice. To further secure the contract, the NFT is transferred to the contract upon calling *createAuctionListing*, which preserves the integrity of the auction and prevents malicious interference. Bidders are also required to send their bid amounts when revealing their bids via *revealBid*, further securing the process.

The auction logic itself is implemented correctly, ensuring that the seller and all participants receive the correct funds and tokens, assuming honest behavior from all parties. However, the reliance of *bid* on the *hasBidder* function to place a new bid introduces a linear increase in gas costs relative to the number of bidders [Listing 1, Listing 2]. This is both a fairness concern for later participants and a potential vulnerability to gas griefing attacks, which is discussed in detail in the following section.

The contract permits either the seller or the winner to finalise the auction, which mitigates, but does not entirely eliminate, the risk of stalling due to inactivity or malicious behavior [Listing 3]. Upon calling *finaliseAuction*, the *resetBids* function automatically resets the highest and second-highest bids as well as the winner’s bid, preventing the possibility of double withdrawals [Listing 4]. Furthermore, the contract adheres to the Checks-Effects-Interactions (CEI) pattern and uses push-based fund transfers rather than pull-based mechanisms, enhancing security [Listing 5].

Compared to my own contract, this contract noticeably does not require a deposit when submitting a bid, enhancing both user-experience (as they do not have to lock away any funds) and gas efficiency (as their payment can be made in one go rather than across two functions). It also means that, should an honest bidder forget to reveal their bid, they will not lose funds as a result. Whilst the choice of not requiring a deposit could result in griefing attacks, the fact that only one bid can be made per address does act as a deterrent to this [Listing 2]. Furthermore, the peer contract relies on fewer parameters, variables, and events, though it does utilise more mappings. This design results in a simpler and more gas-efficient implementation while maintaining the core functionality of a Vickrey auction. The *resetBids* and *transferFund* functions are marked as internal, restricting their usage to within the contract and thereby preventing potential external exploitation.

5.3 Potential Vulnerabilities

Similarly to my contract, this peer contract exhibits a minor vulnerability to front-running attacks during the reveal phase due to the priority given to early revealers in the case of tied bids. However, there are two larger vulnerabilities specific to this design. The first vulnerability arises from the seller’s control over the start of the reveal period [Listing 6]. In this contract, the reveal period only starts once the seller calls the *completeAuction* function, instead of automatically when the bidding period ends. The function must set the reveal period to a duration between 30 seconds and 24 hours and can only be called by the seller. A malicious seller could exploit this design by delaying the call to *completeAuction* in order to stall the auction or by setting a very short reveal period that prevents bidders from revealing in time. This allows the seller to unfairly reclaim the NFT and potentially sell it elsewhere, effectively undermining the fairness of the auction.

The second major vulnerability is related to griefing attacks, which can occur in two primary ways. First, if a seller colludes with a bidder, the attacker can submit a bid guaranteed to be the highest. Both the seller and the colluding bidder could then refuse to call *finaliseAuction*, which is required to finalise the auction. Because the *getrefund* function can only be called after the auction is finalised, all other bidders’ ETH sent during the reveal phase becomes locked indefinitely [Listing 7]. Although this attack is costly for the colluding bidder, who must submit a high enough bid to win, it effectively freezes funds and stalls the auction, creating a severe economic disadvantage for other participants.

Second, a gas-based griefing attack is possible due to the implementation of the *hasBidder* function [Listing 1], used in the *bid* function. This function iterates over the entire *listingBidders* array to check whether a bidder has already participated, giving it a complexity of $O(n)$ per bid, where n is the number of bidders. An attacker could exploit this by creating many Sybil addresses or colluding with other bidders, submitting many bids to artificially inflate the array. As a result, the gas cost for subsequent honest bidders increases linearly with the number of previously registered addresses, potentially causing transactions to fail due to exceeding gas limits or making bidding prohibitively expensive. This undermines the integrity and accessibility of the auction mechanism.

In conclusion, this contract effectively enforces a Vickrey auction in a fair and gas-efficient manner. Whilst certain vulnerabilities, such as the linear gas cost for bids and potential reliance on seller activity, may limit its scalability in auctions with many participants, it remains an intuitive and practical solution for small to medium-sized auctions. Its simplicity, fairness, and gas efficiency make it suitable for running multiple simultaneous auctions while maintaining integrity and usability.

6 Contract Addresses and Relevant Hashes

My contract: 0x8ce9722e5e42ed1ffe534a8acb79fd6f9f100732

Peer’s Contract: 0xf1915efb76b3a7c08d9b9e1190f939cf9adf03d8

Some Hashes of Peer Contract Interaction:

Seller starting auction: 0xb069530573852b6ea2212ad27646efb08723993a139c7b59c4dbb161cc5aad1a

Bidder making bid: 0xb1ee6d651d17c54a64e0275e2a71b39fb25a26edee775d83b4363949e25e0105

Seller finalising auction: 0x29a3e1e2e3638836d72189ad6e972f4efed4c729f339badf87425a3267a509a8

References

- [1] Ulle Endriss, *Game Theory Lecture Slides – Lecture 9*, University of Amsterdam, 2025. <https://staff.science.uva.nl/u.endriss/teaching/game-theory/slides/gt9.pdf>

A Appendix: Test Auction Gas Costs

Table 2: Auction 1: Functions, Callers, and Gas Execution Costs

Function	Caller	Notes	Gas Execution Cost
<i>createAuction</i>	A	Reserve = 10 gwei, Deposit = 5 gwei	235,188
<i>placeSealedBid</i>	B	Bid = 8 gwei	97,879
<i>placeSealedBid</i>	C	Bid = 10 gwei	80,779
<i>placeSealedBid</i>	D	Bid = 15 gwei	80,779
<i>placeSealedBid</i>	E	Bid = 20 gwei	80,779
<i>placeSealedBid</i>	F	Bid = 30 gwei	80,779
<i>reveal</i>	C		89,690
<i>reveal</i>	B		47,315
<i>reveal</i>	D		75,390
<i>endAuction</i>	A		19,240
<i>claimForfeitedBatch</i>	A	Claims for E	57,826
<i>recoverUnclaimedDeposit</i>	F		51,546
<i>withdraw</i>	B		27,179
<i>withdraw</i>	C		27,179
<i>withdraw</i>	D		29,481
<i>claimNFT</i>	D		62,598

Table 3: Auction 1: Functions, Callers, and Gas Execution Costs

Function	Caller	Notes	Gas Execution Cost
<i>createAuction</i>	A	Reserve = 10 gwei, Deposit = 5 gwei	218,088
<i>placeSealedBid</i>	B	Bid = 20 gwei	97,879
<i>placeSealedBid</i>	C	Bid = 20 gwei	80,779
<i>placeSealedBid</i>	D	Bid = 20 gwei	80,779
<i>placeSealedBid</i>	E	Bid = 20 gwei	80,779
<i>placeSealedBid</i>	F	Bid = 20 gwei	80,779
<i>endAuction</i>	B		36,330
<i>claimForfeitedBatch</i>	A	claims for B,C,D,E,F	190,062
<i>reclaimNFT</i>	A		66,912

B Appendix: Key Code Snippets from Peer Contract

```
1 function hasBidder(uint256 listingId, address bidder) internal view returns (bool) {
2     address[] memory bidders = listingBidders[listingId];
3     for (uint i = 0; i < bidders.length; i++) {
4         if (bidders[i] == bidder) return true;
5     }
6     return false;
7 }
```

Listing 1: hasBidder Function (Potential Gas Griefing Vulnerability)

```
1 function bid(uint256 listingId, bytes32 hashedBid) public {
2     // Ensure bidder has not already participated
3     require(!hasBidder(listingId, msg.sender), "You have already placed a bid");
4
5     // Record the bidder
6     listingBidders[listingId].push(msg.sender);
7
8     // Store the hashed bid
9     bids[listingId][msg.sender] = hashedBid;
10
11     emit BidPlaced(listingId, msg.sender);
12 }
```

Listing 2: bid Function (Single-Bid Enforcement Using hasBidder)

```
1 function finaliseAuction(uint256 listingId) public nonReentrant {
2     Listing storage listing = listings[listingId];
3     require(listing.status == STATUS_OPEN, "Auction already finalised");
4     require(listing.auctionEnded, "Auction not ended yet");
5     require(block.timestamp > listing.revealEndAt, "Reveal period not yet over");
6
7     listing.status = STATUS_DONE;
8     IERC721 nft = IERC721(listing.nftAddress);
9     uint256 winnerAmount = highestBid[listingId];
10    address winner = highestBidder[listingId];
11    require(msg.sender == listing.seller || msg.sender == winner,
12           "Only seller or winner can finalise auction");
13
14    // transfer NFT and funds, refund excess
15    resetBids(listingId, winner);
16
17    emit AuctionCompleted(listingId, listing.seller, winner, winnerAmount);
18 }
```

Listing 3: finaliseAuction Function (Locks in Results and Uses resetBids)

```
1 function resetBids(uint256 listingId, address winner) internal {
2     bids[listingId][winner] = 0;
3     highestBid[listingId] = 0;
4     highestBidder[listingId] = address(0);
5     secondHighestBid[listingId] = 0;
6 }
```

Listing 4: resetBids Function (Prevents Double Withdrawals)

```
1 function transferFund(address payable to, uint256 amount) internal {
2     if (amount == 0) { return; }
3
4     require(to != address(0), "Cannot transfer to address(0)");
5     (bool transferSent, ) = to.call{value:amount}("");
6     require(transferSent, "Failed to send Ether");
7 }
```

Listing 5: Checks-Effects-Interactions (CEI) Pattern in transferFund

```

1 function completeAuction(uint256 listingId, uint256 revealBidDurationInSeconds) public nonReentrant {
2     require(msg.sender == listing.seller, "Only seller can complete auction");
3
4     listing.auctionEnded = true;
5     listing.revealEndAt = block.timestamp + revealBidDurationInSeconds;
6 }
7
8 function revealBid(uint256 listingId, uint256 bidAmount, string memory secret) public payable
9     nonReentrant {
10     Listing storage listing = listings[listingId];
11     require(listing.auctionEnded, "Auction not ended yet");
12
13     // ...reveal logic...
14 }

```

Listing 6: Reveal Period Logic in completeAuction and revealBid

```

1 function getRefund(uint256 listingId) public nonReentrant {
2     Listing storage listing = listings[listingId];
3
4     // Ensure auction has been finalised
5     require(listing.status == STATUS_DONE, "Auction not finalised yet");
6
7     uint256 refundAmount = bids[listingId][msg.sender];
8     require(refundAmount > 0, "No refund available");
9
10    // Reset the bidder's amount before transferring to prevent reentrancy
11    bids[listingId][msg.sender] = 0;
12    (bool sent, ) = msg.sender.call{value: refundAmount}("");
13    require(sent, "Refund failed");
14
15    emit RefundClaimed(listingId, msg.sender, refundAmount);
16 }

```

Listing 7: getRefund Function Restricted to After Auction Finalisation

C Appendix: Contract Code

```
1
2 // SPDX-License-Identifier: MIT
3 pragma solidity ^0.8.20;
4
5 import "@openzeppelin/contracts/token/ERC721/IERC721.sol";
6 import "@openzeppelin/contracts/security/ReentrancyGuard.sol"; // protects against reentrancy attacks
7
8
9 /// @title VickreyAuction
10 /// @author Ashe Raymond-Barker
11 /// @notice This contract conducts a sealed bid Vickrey Auction
12 contract VickreyAuction1 is ReentrancyGuard {
13
14     /// STRUCTS
15
16     // represents a single auction with all relevant parameters
17     struct Auction {
18         address seller; // address of the seller
19         IERC721 nft; // address of the NFT collection
20         uint256 tokenId; // unique ID for specific nft
21         uint256 reservePrice; // minimum auction price
22         uint256 endOfBidding; // length of bidding process
23         uint256 endOfRevealing; // length of revealing process
24         uint256 forfeitClaimDeadline; // time limit in which seller has to claim forfeited funds
25         uint256 initialDeposit; // necessary deposit to place a bid
26
27         uint256 highestBid; // value of current highest bid
28         uint256 secondHighestBid; // value of current second highest bid
29         address highestBidder; // address of current highest bidder
30
31         bool auctionEnded; // whether auction has been finalized
32         bool nftClaimed; // whether nft has already been claimed
33         bool sellerWithdrawn; // whether seller has withdrawn their payment from auction
34
35         mapping(address => bool) revealed; // whether a bidder has revealed their bid
36         mapping(address => uint256) deposit; // amount deposited by bidder (including top-ups)
37         mapping(address => bytes32) hashedBid; // hashedbid made by bidder
38         mapping(address => bool) forfeited; // whether bidder's deposit has been forfeited
39         address[] bidders; // list of addresses who placed a sealed bid
40     }
41
42     // struct for users to get auction info whilst it is still ongoing
43     struct AuctionInfo {
44         address seller;
45         address nftContract;
46         uint256 tokenId;
47         uint256 initialDeposit;
48         uint256 endOfBidding;
49         uint256 endOfRevealing;
50         bool auctionEnded;
51     }
52
53     // struct for users to get results of auction upon completion
54     struct AuctionResults {
55         address seller;
56         address nftContract;
57         uint256 tokenId;
58         uint256 reservePrice;
59         uint256 initialDeposit;
60         uint256 endOfBidding;
61         uint256 endOfRevealing;
62         uint256 highestBid;
63         address highestBidder;
64         uint256 secondHighestBid;
65         bool nftClaimed;
66     }
67
68
69     // assign each auction a unique auction ID
70     uint256 public auctionCount = 0;
71     mapping(uint256 => Auction) private auctions; // private because solidity cannot return mappings
72     // inside "Auction" struct
73
74
75     /// EVENTS
76
```

```

77     event AuctionCreated(uint256 indexed auctionId, address indexed seller, address nftContract, uint256
        tokenId, uint256 reservePrice, uint256 initialDeposit, uint256 endOfBidding, uint256
        endOfRevealing); // a new auction has been created
78     event BidPlaced(uint256 indexed auctionId, address indexed bidder); // a bid has been placed
79     event BidRevealed(uint256 indexed auctionId, address indexed bidder, uint256 amount); // a bid has
        been revealed
80     event AuctionEnded(uint256 indexed auctionId, address indexed winner, uint256 highestBid, uint256
        secondHighestBid); // an auction has ended
81     event Withdrawn(uint256 indexed auctionId, address indexed user, uint256 amount); // funds have been
        withdrawn
82     event NFTClaimed(uint256 indexed auctionId, address indexed winner, uint256 tokenId); // NFT has
        been claimed by winner
83     event NFTReclaimed(uint256 indexed auctionId, address indexed seller, uint256 tokenId); // NFT has
        been reclaimed by seller
84     event DepositForfeited(uint256 indexed auctionId, address indexed bidder, uint256 amount); // a
        deposit has been forfeited
85     event ForfeitedClaimed(uint256 indexed auctionId, address indexed seller, uint256 totalAmount); // a
        forfeited deposit has been claimed by seller
86     event UnclaimedDepositRecovered(uint256 indexed auctionId, address indexed bidder, uint256 amount);
        // a forfeited deposit has been reclaimed by the bidder
87
88
89
90     /// MODIFIERS
91
92     // only seller can call this function
93     modifier onlySeller(uint256 auctionId) {
94         require(msg.sender == auctions[auctionId].seller, "Only seller can call function");
95         _;
96     }
97
98     // only callable during the bidding period
99     modifier onlyDuringBidding(uint256 auctionId) {
100         require(block.timestamp <= auctions[auctionId].endOfBidding, "Bidding period ended");
101         _;
102     }
103
104     // only callable during the reveal period
105     modifier onlyDuringReveal(uint256 auctionId) {
106         require(block.timestamp > auctions[auctionId].endOfBidding, "Bidding period not ended");
107         require(block.timestamp <= auctions[auctionId].endOfRevealing, "Reveal period ended");
108         _;
109     }
110
111     // only callable after the reveal period
112     modifier onlyAfterReveal(uint256 auctionId) {
113         require(block.timestamp > auctions[auctionId].endOfRevealing, "Reveal period not ended");
114         _;
115     }
116
117     // only allow function if NFT has not been claimed yet
118     modifier onlyIfNFTNotClaimed(uint256 auctionId) {
119         require(!auctions[auctionId].nftClaimed, "NFT already claimed");
120         _;
121     }
122
123     // only callable after auction has officially been ended/finalised
124     modifier onlyAfterAuctionEnded(uint256 auctionId) {
125         require(auctions[auctionId].auctionEnded, "Auction not ended yet");
126         _;
127     }
128
129     // checks auction you are calling function for exists
130     modifier auctionExists(uint256 auctionId) {
131         require(auctions[auctionId].seller != address(0), "Auction does not exist");
132         _;
133     }
134
135
136     // Only callable while the seller is allowed to claim forfeited deposits
137     modifier onlyDuringClaim(uint256 auctionId) {
138         require(block.timestamp <= auctions[auctionId].forfeitClaimDeadline, "Claim period ended");
139         require(block.timestamp > auctions[auctionId].endOfRevealing, "Claim period not started");
140         _;
141     }
142
143     // Only callable after the sellers claim window has closed
144     modifier onlyAfterClaim(uint256 auctionId) {
145         require(block.timestamp > auctions[auctionId].forfeitClaimDeadline, "Claim period not ended");
146         _;
147     }

```



```

148
149
150
151 /// FUNCTIONS
152
153
154 /// @notice Creates a new auction
155 /// @param _nft address of the NFT collection
156 /// @param _tokenId unique ID for specific nft
157 /// @param _reservePrice minimum auction price
158 /// @param _biddingPeriod length of bidding period
159 /// @param _revealingPeriod length of revealing period
160 /// @param _initialDeposit deposit to bid in auction
161 function createAuction(IERC721 _nft, uint256 _tokenId, uint256 _reservePrice, uint32 _biddingPeriod,
162     uint32 _revealingPeriod, uint256 _initialDeposit)
163     external {
164
165         require(address(_nft) != address(0), "NFT contract requires address");
166         require(_biddingPeriod >= 180 && _revealingPeriod >= 180, "Bid & reveal periods too short");
167         require(_biddingPeriod <= 4 * _revealingPeriod, "Needs bid period <= 4 * reveal period");
168         require(_reservePrice > 0, "Reserve price must be > 0");
169         require(_nft.ownerOf(_tokenId) == msg.sender, "You do not own this NFT");
170         require(_initialDeposit > 0, "Initial deposit must be > 0");
171         require(_reservePrice >= _initialDeposit, "Reserve must be >= deposit");
172
173         uint256 auctionId = auctionCount++; // gives auction an ID value
174
175         // initialise parameters for specific auction
176         Auction storage a = auctions[auctionId]; // shortened auction identifier
177         a.seller = msg.sender;
178         a.nft = _nft;
179         a.tokenId = _tokenId;
180         a.reservePrice = _reservePrice;
181         a.initialDeposit = _initialDeposit;
182         a.endOfBidding = block.timestamp + _biddingPeriod;
183         a.endOfRevealing = a.endOfBidding + _revealingPeriod;
184         a.forfeitClaimDeadline = a.endOfRevealing + _revealingPeriod; // same amount of time as reveal
185         period
186
187         // transfer NFT to contract
188         _nft.transferFrom(msg.sender, address(this), _tokenId);
189
190         emit AuctionCreated(auctionId, msg.sender, address(_nft), _tokenId, _reservePrice,
191             _initialDeposit, a.endOfBidding, a.endOfRevealing);
192     }
193
194 /// @notice Bidders place sealed bid accompanied by deposit
195 /// @param auctionId auction bidder wants to participate in
196 /// @param hash encrypted bid (computed off-chain)
197 function placeSealedBid(uint256 auctionId, bytes32 hash)
198     external payable auctionExists(auctionId) onlyDuringBidding(auctionId) {
199
200         Auction storage a = auctions[auctionId];
201         require(msg.sender != a.seller, "Seller cannot bid");
202         require(msg.value == a.initialDeposit, "Must send initial deposit");
203         require(a.hashedBid[msg.sender] == bytes32(0), "Bid already placed");
204
205         a.hashedBid[msg.sender] = hash; // maps hashed bid to bidder
206         a.deposit[msg.sender] = msg.value; // maps deposit value to bidder
207         a.bidders.push(msg.sender); // adds bidder to list of all bidders
208
209         emit BidPlaced(auctionId, msg.sender); // deposit value is same for everyone so is not emitted
210     }
211
212 /// @notice Bidders reveal their bid after bidding period ends
213 /// @param auctionId auction bidder participated in
214 /// @param amount value bid by bidder
215 /// @param nonce secret number chosen by bidder to compute hash
216 function reveal(uint256 auctionId, uint256 amount, uint256 nonce)
217     external payable onlyDuringReveal(auctionId) auctionExists(auctionId) {
218
219         Auction storage a = auctions[auctionId];
220         require(!a.revealed[msg.sender], "Already revealed");
221         require(a.hashedBid[msg.sender] != bytes32(0), "No bid placed");
222         require(keccak256(abi.encode(msg.sender, amount, nonce)) == a.hashedBid[msg.sender], "Invalid
223             reveal"); // commitment unique per bidder, no encoding ambiguity, negligible gas overhead
224
225         if (amount > a.deposit[msg.sender]) {

```

```

224         uint256 remaining = amount - a.deposit[msg.sender]; // Calculate the remaining amount to be
225             sent
226         require(msg.value == remaining, "Send remaining bid amount");
227         a.deposit[msg.sender] += msg.value; // update deposit amount
228     } else {
229         require(msg.value == 0, "No extra ETH needed");
230     }
231
232     // mark sender as having revealed
233     a.revealed[msg.sender] = true;
234
235     // updates highest and second highest bids with priority for first to reveal in the case of ties
236     if (amount > a.highestBid && amount >= a.reservePrice) {
237         a.secondHighestBid = a.highestBid;
238         a.highestBid = amount;
239         a.highestBidder = msg.sender;
240     } else if (amount > a.secondHighestBid && amount >= a.reservePrice) {
241         a.secondHighestBid = amount;
242     }
243
244     emit BidRevealed(auctionId, msg.sender, amount);
245 }
246
247 /// @notice Seller or bidder marks auction as ended after reveal period finishes
248 /// @param auctionId auction seller or bidder participated in
249 function endAuction(uint256 auctionId)
250     external auctionExists(auctionId) onlyAfterReveal(auctionId) {
251
252     Auction storage a = auctions[auctionId];
253     require(!a.auctionEnded, "Auction already ended");
254     a.auctionEnded = true; // mark auction as finalized
255
256     if (a.secondHighestBid >= a.reservePrice) {
257         emit AuctionEnded(auctionId, a.highestBidder, a.highestBid, a.secondHighestBid);
258     } else {
259         emit AuctionEnded(auctionId, a.highestBidder, a.highestBid, a.reservePrice);
260     }
261 }
262
263
264 /// @notice Seller and bidder withdraws funds after auction has ended
265 /// @param auctionId auction seller or bidder participated in
266 function withdraw(uint256 auctionId)
267     external nonReentrant auctionExists(auctionId) onlyAfterAuctionEnded(auctionId) {
268
269     Auction storage a = auctions[auctionId];
270     uint256 value; // amount seller or bidder must receive
271
272     // case 1: Seller
273     if (msg.sender == a.seller) {
274         require(!a.sellerWithdrawn, "Already withdrawn");
275         a.sellerWithdrawn = true; // prevents seller attempting multiple withdrawals
276         require(a.highestBid >= a.reservePrice, "No valid bids");
277         if (a.secondHighestBid >= a.reservePrice) {
278             value = a.secondHighestBid;
279         } // case where two valid bids are made above the reserve price
280         else {
281             value = a.reservePrice;
282         } // case where zero or one valid bid is made above reserve price
283     }
284
285     // Case 2: Winner
286     else if (msg.sender == a.highestBidder) {
287         if (a.secondHighestBid >= a.reservePrice) {
288             value = a.deposit[msg.sender] - a.secondHighestBid;
289         } // case where two valid bids are made above the reserve price
290         else {
291             value = a.deposit[msg.sender] - a.reservePrice;
292         } // case where one valid bid is made above reserve price
293     }
294
295     // Case 3: Losing Bidders
296     else {
297         require(a.revealed[msg.sender], "Unrevealed bids forfeited"); // those who didn't reveal
298             bids lose their deposit
299         value = a.deposit[msg.sender];
300     }
301
302     require(value > 0, "No funds to withdraw"); // stops wasting gas on a zero withdrawal
303     a.deposit[msg.sender] = 0; // reset deposit to prevent bidders attempting multiple withdrawals

```

```

303
304     (bool ok, ) = msg.sender.call{value: value}("");
305     require(ok, "Transfer failed");
306
307     emit Withdrawn(auctionId, msg.sender, value);
308 }
309
310
311 /// @notice Winner can claim NFT after auction ends
312 /// @param auctionId auction bidder participated in
313 function claimNFT(uint256 auctionId)
314     external nonReentrant auctionExists(auctionId) onlyAfterAuctionEnded(auctionId)
315     onlyIfNFTNotClaimed(auctionId) {
316
317     Auction storage a = auctions[auctionId];
318     require(msg.sender == a.highestBidder, "you are not highest bidder"); // Only the highest bidder
319     can claim the NFT
320
321     a.nftClaimed = true;
322     a.nft.safeTransferFrom(address(this), a.highestBidder, a.tokenId);
323
324     emit NFTClaimed(auctionId, a.highestBidder, a.tokenId);
325 }
326
327 /// @notice Seller can reclaim NFT after auction ends if there were no valid bids
328 /// @param auctionId auction seller participated in
329 function reclaimNFT(uint256 auctionId)
330     external nonReentrant auctionExists(auctionId) onlySeller(auctionId)
331     onlyAfterAuctionEnded(auctionId) onlyIfNFTNotClaimed(auctionId) {
332
333     Auction storage a = auctions[auctionId];
334     require(a.highestBid < a.reservePrice, "There was a valid bid");
335
336     a.nftClaimed = true;
337     a.nft.safeTransferFrom(address(this), a.seller, a.tokenId);
338
339     emit NFTReclaimed(auctionId, a.seller, a.tokenId);
340 }
341
342 /// @notice Seller can Claim forfeited deposits in batches
343 /// @param auctionId auction seller participated in
344 /// @param biddersToClaim list of bidders to claim from
345 /// @param maxBatch maximum number of bidders to claim from in one instance for gas limit reasons
346 function claimForfeitedBatch(uint256 auctionId, address[] calldata biddersToClaim, uint16 maxBatch)
347     external auctionExists(auctionId) onlySeller(auctionId) nonReentrant
348     onlyAfterAuctionEnded(auctionId) onlyDuringClaim(auctionId) {
349
350     Auction storage a = auctions[auctionId];
351     uint256 total; // amount to transfer to seller
352     uint16 count; // to ensure batch size limit is not violated
353
354     for (uint256 i = 0; i < biddersToClaim.length && count < maxBatch; ++i) {
355         address b = biddersToClaim[i];
356         // if bidder failed to reveal and hasn't already forfeited deposit
357         if (!a.revealed[b] && !a.forfeited[b] && a.deposit[b] > 0) {
358             uint256 amt = a.deposit[b];
359             a.deposit[b] = 0;
360             a.forfeited[b] = true; // zeros deposit and marks deposit as forfeited before adding to
361             the total
362             total += amt;
363             count++;
364             emit DepositForfeited(auctionId, b, amt); // records which bidder has forfeited their
365             deposit
366         }
367     }
368
369     require(total > 0, "No deposits to claim"); // prevents wasting gas on a zero value claim
370     (bool ok, ) = a.seller.call{value: total}("");
371     require(ok, "Transfer failed");
372
373     emit ForfeitedClaimed(auctionId, a.seller, total);
374 }
375
376 /// @notice Bidders who failed to reveal can claim their deposit back after seller's claim period
377 ends
378 /// @param auctionId auction bidder participated in
379 function recoverUnclaimedDeposit(uint256 auctionId)

```

```

376     external auctionExists(auctionId) nonReentrant onlyAfterAuctionEnded(auctionId)
377         onlyAfterClaim(auctionId) {
378
379         Auction storage a = auctions[auctionId];
380         require(!a.revealed[msg.sender], "Bid was revealed");
381         require(!a.forfeited[msg.sender], "Deposit already forfeited");
382         uint256 amount = a.deposit[msg.sender];
383         require(amount > 0, "Nothing to recover");
384
385         a.deposit[msg.sender] = 0;
386         a.forfeited[msg.sender] = true;
387
388         (bool ok, ) = msg.sender.call{value: amount}("");
389         require(ok, "Transfer failed");
390
391         emit UnclaimedDepositRecovered(auctionId, msg.sender, amount);
392     }
393
394     /// @notice Computes the hash of a bid for use in a Vickrey auction commitment
395     /// @param bidder the address of the bidder using function
396     /// @param amount the bid made by the bidder
397     /// @param nonce secret number chosen by bidder
398     /// @dev This function is pure and can be called off-chain to generate the hashed bid.
399     ///       The hash is computed using 'abi.encode(bidder, amount, nonce)'.
400     ///       It does not interact with contract storage or state.
401     function computeHash(address bidder, uint256 amount, uint256 nonce) external pure returns (bytes32) {
402         return keccak256(abi.encode(bidder, amount, nonce));
403     }
404
405
406
407     /// GETTER FUNCTIONS
408
409     /// @notice Returns key information about a specific auction
410     /// @param auctionId the specific auction that info is required for
411     /// @dev can be called off chain for no gas cost
412     function getAuctionInfo(uint256 auctionId)
413         external view auctionExists(auctionId) returns (AuctionInfo memory info) {
414
415         Auction storage a = auctions[auctionId];
416         info = AuctionInfo({
417             seller: a.seller,
418             nftContract: address(a.nft),
419             tokenId: a.tokenId,
420             initialDeposit: a.initialDeposit,
421             endOfBidding: a.endOfBidding,
422             endOfRevealing: a.endOfRevealing,
423             auctionEnded: a.auctionEnded
424         });
425     }
426
427
428     /// @notice Returns key information and results about a specific auction after it has ended
429     /// @param auctionId the specific auction that info is required for
430     /// @dev can be called off chain for no gas cost
431     function getAuctionResults(uint256 auctionId)
432         external view auctionExists(auctionId) onlyAfterAuctionEnded(auctionId) returns (AuctionResults
433             memory results) {
434
435         Auction storage a = auctions[auctionId];
436         results = AuctionResults({
437             seller: a.seller,
438             nftContract: address(a.nft),
439             tokenId: a.tokenId,
440             reservePrice: a.reservePrice,
441             initialDeposit: a.initialDeposit,
442             endOfBidding: a.endOfBidding,
443             endOfRevealing: a.endOfRevealing,
444             highestBid: a.highestBid,
445             highestBidder: a.highestBidder,
446             secondHighestBid: a.secondHighestBid,
447             nftClaimed: a.nftClaimed
448         });
449     }
450
451     /// @notice Returns the hashed bid submitted by a specific bidder
452     /// @param auctionId auction bidder participated in
453     /// @param bidder address of the specific bidder
454     /// @dev can be called off chain for no gas cost

```

```

455     function getBidHash(uint256 auctionId, address bidder) external view returns (bytes32) {
456         return auctions[auctionId].hashedBid[bidder];
457     }
458
459     /// @notice Returns the total deposit currently held for a bidder
460     /// @param auctionId auction bidder participated in
461     /// @param bidder address of the specific bidder
462     /// @dev can be called off chain for no gas cost
463     /// deposit refers to initial deposit plus top ups
464     function getDeposit(uint256 auctionId, address bidder) external view returns (uint256) {
465         return auctions[auctionId].deposit[bidder];
466     }
467
468     /// @notice Returns whether a bidder has revealed their bid
469     /// @param auctionId auction bidder participated in
470     /// @param bidder address of the specific bidder
471     /// @dev can be called off chain for no gas cost
472     function hasRevealed(uint256 auctionId, address bidder) external view returns (bool) {
473         return auctions[auctionId].revealed[bidder];
474     }
475
476     /// @notice Returns whether a bidders deposit has been forfeited
477     /// @param auctionId auction bidder participated in
478     /// @param bidder address of the specific bidder
479     /// @dev can be called off chain for no gas cost
480     function hasForfeited(uint256 auctionId, address bidder) external view returns (bool) {
481         return auctions[auctionId].forfeited[bidder];
482     }
483
484     /// @notice Returns the list of all bidder addresses for a given auction
485     /// @param auctionId the specific auction that info is required for
486     /// @dev can be called off chain for no gas cost
487     function getBidders(uint256 auctionId) external view returns (address[] memory) {
488         return auctions[auctionId].bidders;
489     }
490 }

```